

scripts/pipeline/phase-04-live-verify-api-app.sh

External user guide (HelixConstitution §11.4.18 companion doc) for the :api-app half of the local build-test-distribute pipeline's FR-008 live-verification phase (specs/002-build-test-distribute-pipeline, task T037).

Classification: project-specific (Lava's :api-app module, its Challenge suite, and this pipeline's evidence layout; the delegate-to-Containers emulator orchestration it reuses is universal per §6.X).

What it does

Takes the real :api-app **debug APK** that phase-01 produced, has the Containers submodule put it onto a real **cold-booted Android emulator running inside a container**, and drives the real Compose UI **boot-and-serve Challenge** against it — the Challenge that starts the on-device API embed and then, as a genuine HTTPS peer, asserts the embed actually answers (/health → 200 + real JSON body; an auth-gated route → 401 without the key; the same route → not-401 with the key the UI displayed).

It is the sibling of scripts/pipeline/phase-04-live-verify-api.sh, which proves the standalone lava-api-go **service** is live. Before this script existed, a green live_verify phase meant only "the Go API is live" and said nothing about :api-app.

Why it is not the same as phase-02's Challenge pass

	phase-02-test-challenge.sh	this script
Scope		the boot-and-serve Challenge only

	phase-02-test-challenge.sh	this script
	the whole discovered Challenge suite, both modules	
Claim	"the test suite is green"	"the shipped artifact, installed on a device, boots and serves"
Evidence dir	<run>/phase-02/...	<run>/phase-04/...
report.json phase	test	live_verify
Runs	before install/boot	after phase-03's install/boot

§6.Z's forensic anchor — a distribute green-lit by a test pass that never exercised the shipped behaviour — is exactly why the two must not be conflated.

Usage

```
scripts/pipeline/phase-04-live-verify-api-app.sh <run_id>
[repo-path]
```

<run_id> must already have a report.json (created by lib/run-report.sh's init_run_report). The script appends one entry to that report under the phase name live_verify — the same name its lava-api-go sibling uses. Both entries legitimately coexist in phases[]; they are the two halves of FR-008 and are told apart by their evidence_dir.

Environment overrides

Variable	Default	Meaning
LAVA_PIPELINE_LIVE_VERIFY_API_APP_TEST_CLASSES	the discovered boot-and-serve Challenge	Comma-separated FQCN list, intersected with what is really on disk — a class named here with

Variable	Default	Meaning
		no matching file is never invented.
LAVA_PIPELINE_LIVE_VERIFY_API_APP_CONTAINER_RUNTIME	podman	podman or docker.
		Hard outer bound on the whole matrix invocation. On expiry the run is killed, surviving containers are reaped, and the real timeout is recorded — never a fabricated pass.
LAVA_PIPELINE_LIVE_VERIFY_API_APP_TIMEOUT_SECONDS	2700 (45m)	Forwarded to the matrix script's --boot-timeout.
LAVA_PIPELINE_LIVE_VERIFY_API_APP_BOOT_TIMEOUT	10m	

Exit codes

Code	Meaning
0	Every Evidence Record is PASS (or an honest SKIPPED) and every record was anti-bluff-validated.
1	A real failure: the run genuinely failed, the two independent sources disagreed, the \$6.AH container proof was absent, the artifact was missing, or a record was REJECTED. Recorded as FAIL in report.json either way.

Code	Meaning
2	Usage/precondition error (missing <code>run_id</code> , absent <code>report.json</code> , missing tool or matrix script, zero discoverable Challenge classes).

Constitutional posture

- **\$6.AH / \$6.AG / \$6.X.** The emulator comes from `scripts/run-api-app-challenge-matrix.sh` → the Containers submodule's `cmd/emulator-matrix` CLI, which boots the emulator process *inside* a podman/docker container. No host-direct emulator, no physical device, no fallback. **The script does not merely trust that:** while the run is in flight, its own background poller samples `<runtime> ps --filter name=lava-emu` and the device list every 10s into `hermetic-script/raw/live-target-proof.log`. If no running emulator container was ever observed, the provenance record is **FAIL** — \$6.AH-compliance is a checked fact here, not a claim.
- **Decoupled Reusable Architecture.** No emulator orchestration is reimplemented; the script is thin glue over the existing matrix script.
- **\$6.R.** No IPv4, host:port, or UUID literal. The image reference is built from the API level chosen at runtime; the Challenge class is *discovered* from the real source files (their own package line + filename), never hardcoded as an FQCN that could drift.
- **\$6.U.** No `sudo/su`.

Which artifact is verified

`api-app/build/outputs/apk/debug/api-app-debug.apk` — the exact path the matrix script passes to the Containers CLI's `--apk` flag and the exact path `phase-01-build-android.sh` publishes as its `api-app-debug` artifact.

Note on the task wording. `tasks.md` T037 says "the T018 debug APK", but T018 is phase-01's `:api-app` **release** step and T017 is the **debug** step. The debug APK is what is meant and what is actually installable, because `androidTest` instrumentation APKs are built and signed against the debug variant — `connectedDebugAndroidTest` cannot drive a release APK.

The script passes `--no-build`, so the matrix script rebuilds nothing; Gradle's own `connectedDebugAndroidTest` still brings the debug + androidTest APKs up to date with the current source before instrumenting. The recorded sha256 is computed **after** the run, so it is the hash of the APK that was really on the device.

AVD scope on a host with partial image availability

The §6.AE.2 gate matrix is API 28/30/34/latest × phone/tablet. The Containers CLI's image preflight aborts the whole run if **any** requested AVD's image is missing, so this script picks the first candidate API level whose image is genuinely present locally (checked with a real `<runtime> image inspect`) and scopes `--avds` to that one phone AVD. When none are cached it still attempts the lowest candidate for real, so the genuine pull failure becomes the run's diagnostic instead of a guess.

This phase is a live-verification of the shipped artifact, not the §6.AE.2 release gate. That full-matrix obligation is unchanged and still owed at tag time.

Evidence Records written

Both land under `<run>/phase-04/`.

1. **hermetic-script / api-app-live-verify-install-and-runner-provenance** — the §6.AH/§6.AG proof: the real sha256 + byte size of the APK, the AVD / API level / diagnostics the Containers attestation recorded, the real container name + image **observed running** by the script's own poller, and confirmation that no physical device was a target. FAILs when the container proof is absent or the attestation reports an install failure.
2. **real-device-challenge / <FQCN>** (one per verified Challenge class) — PASS only when **both** independent sources agree:
 - **Source A:** Gradle's own host-side JUnit XML under `api-app/build/outputs/androidTest-results/connected/**/TEST-*.xml`, freshness-filtered to this run, with no `<failure>`/`<error>`.
 - **Source B:** the Containers attestation row reporting `test_passed=true`.

Disagreement between them is itself a **FAIL** — a matrix runner claiming green while Gradle's own report showed a failure is precisely the bluff FR-004 exists to catch. The `raw_output_ref` file embeds the class's own FALSIFIABILITY REHEARSAL KDoc block **verbatim from its .kt source**, which is also what satisfies `anti-bluff-validate.sh` Rule 4 for this category.

Directory-depth constraint

`finalize_run_report`'s aggregator selects records with `find <phase_dir> -mindepth 2 -maxdepth 2 -name '*.json'`. Any stray `.json` at exactly depth 2 under `phase-04` would be miscounted as an Evidence Record. All raw output — including the matrix runner's own `real-device-verification.json` / `host-preflight.json` — is therefore kept at depth 3 or deeper, under `<category>/raw/`.

Cleanup

After the run the script lists and force-removes any surviving `lava-emu-*` container (recorded in `hermetic-script/raw/cleanup.log`), so an interrupted or partially-torn-down run never leaks an orphaned emulator.

Anti-bluff posture

A blocked run reports **BLOCKED/FAIL with the real error**; it never fabricates or assumes a pass. Specifically:

- Artifact missing → FAIL, naming the path.
- Outer timeout hit → FAIL, quoting the real matrix log tail.
- No container observed → FAIL, quoting the real preflight and log tail.
- Neither source recorded the class → FAIL, quoting what *was* present.
- Sources disagree → FAIL, quoting both verdicts.

Related

- `scripts/pipeline/phase-04-live-verify-api.sh` — the `lava-api-go` half.

- `scripts/pipeline/phase-02-test-challenge.sh` — the build-time Challenge pass.
- `scripts/run-api-app-challenge-matrix.sh` — the delegated matrix entry point.
- `scripts/pipeline/lib/evidence.sh`, `lib/anti-bluff-validate.sh`, `lib/run-report.sh` — the shared pipeline libraries.